

DEVELOPER & SECURITY

ShadowTalk AI Developer Studio & Security Vault

In-browser virtual Node.js OS via StackBlitz WebContainer, Monaco Editor, PBKDF2 + AES-GCM encryption, Stealth Network Guard, and Cloud Security Rules.

VIRTUAL RUNTIME

StackBlitz@webcontainer/api

CODE EDITOR

Monaco Editor (VS Code Engine)

CRYPTOGRAPHIC STANDARD

AES-256-GCM + PBKDF2 (100k iters)

NETWORK GUARD

Active Fetch/XHR Interceptor

1 Personal IDE & WebContainer Runtime

ShadowTalk features a complete in-browser software development suite (`src/components/chat/PersonalIDE.tsx` , 103 KB) and `src/lib/webcontainer/engine.ts` . Developers can write, compile, run, and preview full-stack Node.js projects without installing local runtimes:

WebContainer Virtual Operating System

Using WebAssembly and browser `SharedArrayBuffer` primitives, StackBlitz WebContainer boots a headless Node.js virtual kernel directly in browser memory:

- **File System Tree:** The workspace converts project `FileNode[]` structures into a hierarchical `FileSystemTree` mounted via `instance.mount(tree)` .
- **Command Execution:** Spawns processes (e.g. `npm install` , `node index.js`) via `instance.spawn()` , streaming stdout and stderr to a terminal interface.
- **Live Dev Server Preview:** Detects open server ports (e.g. port 3000 or 5173) and displays an embedded, hot-reloading iframe preview inside the workspace canvas.

Monaco Editor Integration

The code editor leverages Microsoft's Monaco Editor engine with support for multi-tab navigation, syntax highlighting, diff viewing, line bookmarking, and automatic formatting across TypeScript, JavaScript, Python, HTML, CSS, SQL, and JSON.

2 End-to-End Cryptography (E2EE)

User privacy in ShadowTalk is enforced at the mathematical level via `src/lib/e2e-encryption.ts` using standard Web Crypto API primitives:

Key Derivation (PBKDF2)

The user's master vault password never leaves the device. Keys are derived using PBKDF2 with **100,000 iterations** of SHA-256 and a cryptographically secure 16-byte random salt generated via `crypto.getRandomValues()` .

Authenticated Encryption (AES-GCM)

Plaintext payloads are encrypted with **AES-256-GCM** using a unique 12-byte initialization vector (IV) per message. The ciphertext and authentication tag ensure tamper detection and confidentiality on cloud servers.

3 Stealth Kill Switch & Network Guard

For sensitive operational security (OPSEC), ShadowTalk includes an active network kill switch in `src/lib/stealthNetworkGuard.ts` :

Active Network Monkey-Patching

When Stealth Mode is engaged via `StealthKillSwitchContext.tsx` :

- `window.fetch` and `XMLHttpRequest.prototype.open` are dynamically overridden.
- Outgoing URLs are checked against a strict whitelist allowing only `blob:` , `data:` , and verified local origins (`localhost` , `127.0.0.1`).
- Any outbound telemetry, cloud sync, or external API call is immediately terminated, triggering a blocked request counter in the UI.

4 Biometric Authentication & Threat Intelligence

Access control to encrypted local vaults is secured via modern web authentication standards:

- **WebAuthn Biometrics (useBiometricAuth.ts):** Integrates platform authenticators (Face ID, Touch ID, Windows Hello) to unlock local AES keys without retyping master passwords.
- **ShadowSpectre Cyber Suite:** Implemented in `src/lib/cyber/shadowspectre/`. Includes specialized security reasoning personas capable of parsing vulnerability reports, auditing HTTP response headers, triaging CVE alerts, and validating defensive network policies.

5 Cloud Security Rules & Access Controls

Cloud persistence in Firebase is fortified through declarative rules enforced at the database kernel level:

Firestore Security Rules (firestore.rules)

```
service cloud.firestore {
  match /databases/{database}/documents {
    function signedIn() { return request.auth != null; }
    function isAdmin() {
      return signedIn() &&
        exists(/databases/{database}/documents/user_roles/{request.auth.uid}) &&
        get(/databases/{database}/documents/user_roles/{request.auth.uid}).data.role == 'admin';
    }
    function ownsExisting() { return signedIn() && resource.data.user_id == request.auth.uid; }
    function ownsIncoming() { return signedIn() && request.resource.data.user_id == request.auth.uid; }

    // User Roles: Admin managed only
    match /user_roles/{uid} {
      allow read: if signedIn() && (request.auth.uid == uid || isAdmin());
      allow write: if isAdmin();
    }

    // Default per-user isolation
    match /{collection}/{doc} {
      allow read: if ownsExisting() || isAdmin();
      allow create: if ownsIncoming();
      allow update, delete: if (ownsExisting() && ownsIncoming()) || isAdmin();
    }
  }
}
```

Cloud Storage Security Rules (storage.rules)

File uploads are restricted to `/users/{uid}/*` folders. Uploads enforce ownership verification (`request.auth.uid == uid`), maximum payload caps of 50 MB, and explicit MIME-type filtering to prevent executable payload uploads.